Report for ActTogether Social Networking Platform

Introduction

For this assignment, I chose to build a social networking platform called ActTogether. This topic was selected from the list provided in the brief, which asked for a platform where users can create profiles, connect with friends, share posts, upload photos/videos, and send messages. The platform is built using ASP.NET Core MVC with SQL Server as the database, following the architectural patterns demonstrated in the sample applications developed during class, including RazorApp, Product API, and AppWebApi.

Project Overview

ActTogether is designed as a social networking platform specifically for actors. Users can create profiles with bio information and profile pictures, send and accept friend requests, share video performances, exchange private messages, and give feedback on each other's work. The platform includes an admin dashboard for managing user reports and contact messages. The feed shows posts only from confirmed friends, creating a private environment for sharing work-in-progress performances.

The platform was intentionally designed as a private social network rather than a public one. This approach matches the intended use case - actors sharing audition tapes with friends for constructive feedback - while significantly reducing development complexity. By limiting the feed to confirmed friends, the application avoids the need for content recommendation algorithms, complex permission systems, public-facing content moderation, and features like likes or public comments that would be required for a more typical social network.

Architecture Design

Separation of Concerns

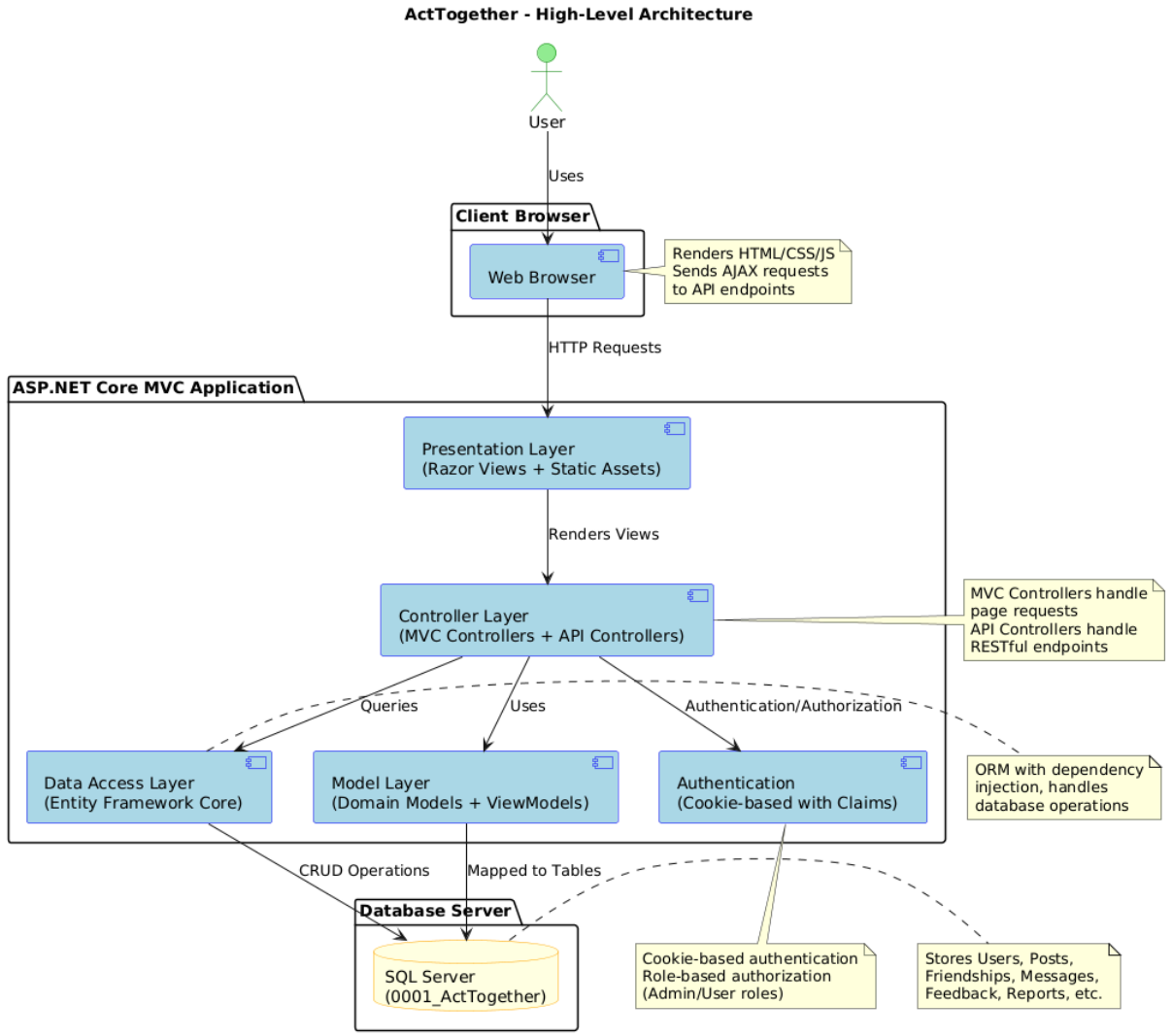
The application follows the Model-View-Controller (MVC) pattern with clear separation of concerns. Models represent database entities and are located in the Models folder. Controllers handle HTTP requests, coordinate business logic, and return views. Views use Razor syntax to render HTML with server-side data binding. This structure ensures a maintainable and scalable codebase.

Data Access Layer

Entity Framework Core serves as the ORM, mapping C# classes to SQL Server tables. The database context defines DbSets for each entity, and connection strings are configured in appsettings.json. Controllers receive the AppDbContext through constructor injection using dependency injection, keeping the data access layer cleanly separated from the business logic.

Architecture Diagram

The architecture diagram below illustrates how the application layers connect. The user interacts with the browser, which sends HTTP requests to the Presentation Layer (Razor Views). The Controller Layer handles both MVC page requests and RESTful API endpoints. Controllers use the Model Layer for data structures, the Data Access Layer (Entity Framework Core) for database operations, and the Authentication layer for cookie-based identity with Claims. The Data Access Layer communicates with the SQL Server database for all CRUD operations.



Backend Implementation

Web APIs and CRUD Operations

The application implements RESTful Web API controllers alongside the MVC controllers. API controllers such as UsersApiController, PostsApiController, FriendshipsApiController, and MessagesApiController each include GET, POST, PUT, and DELETE operations. For example, the UsersApiController provides endpoints for retrieving user profiles, updating user information, and deleting users. These endpoints return appropriate HTTP status codes such as 200 OK, 404 Not Found, and 204 No Content. API documentation is available via Swagger at https://localhost:7178/swagger/index.html when running the application.

Authentication and Authorization

Authentication uses cookie-based authentication with Claims. Users are identified via a ClaimsIdentity containing their ID, username, and role, issued as a cookie on login. Passwords are hashed using BCrypt before storage. Role-based authorization protects admin-only areas, ensuring only users with the Admin role can access the Admin Dashboard. User identity is accessed through ClaimsPrincipal to get the current user's ID and role information.

Data Validation

Both client-side and server-side validation are implemented. The registration form includes validation for required fields, email format, password length, and bio length. Server-side validation uses data annotations on ViewModels. This ensures data integrity even if client-side validation is bypassed.

MVC and Web API Integration

The MVC views communicate with Web API endpoints using AJAX calls. For example, deleting a post uses a JavaScript function that calls the DELETE endpoint without requiring a page reload. The messaging inbox updates unread message counts through API calls. This seamless integration between the UI and API layer enables dynamic user interactions without full page reloads.

User Interface

Page Structure

The application includes the following pages:

1. **Login Page** – Functions as a landing page with a split-screen layout; one half provides the platform's pitch and information about what ActTogether offers to prospective users, while the other half allows authentication with username and password
2. **Register Page** – New users can create accounts with profile information
3. **Feed (Home Page)** – Main page after login showing audition tapes from friends
4. **Interactive Pages** – Profiles, Friends, Friend Requests, Search, Messages, and My Videos for managing the social networking features
5. **Contact Us** – Form for authenticated users to send messages to administrators
6. **Admin Dashboard** – Separate section for administrators to manage contact messages and user reports

Responsive Design

The user interface uses Bootstrap 5 with the Sketchy theme from Bootswatch for consistent styling and responsive design. The Sketchy theme was chosen over alternatives like Quartz for its creative character while maintaining a simple black and white color scheme that keeps the interface clean and focused. The navigation bar shows different options based on authentication status and user role. The layout adapts to different screen sizes using Bootstrap's grid system, with navigation collapsing into a hamburger menu on smaller screens.

Accessibility

The application incorporates accessibility features through Bootstrap 5's built-in support. This includes semantic HTML structure, ARIA attributes on navigation elements, keyboard-friendly focus management, and responsive design that adapts to different devices and screen sizes.

Navigation and User Experience

Navigation is consistent throughout the application. Authenticated users see links to Feed, Friends, Requests, Search, Messages, My Videos, and Contact Us. Admin users additionally see a link to the Admin Dashboard. This conditional navigation ensures users only see options relevant to their role and authentication status.

Client-Side Scripting

JavaScript is used effectively to enhance the user experience. AJAX calls allow deleting posts, updating unread message badges, and handling form submissions without full page reloads. This improves interactivity while maintaining server-side validation.

Core Features

User Profiles and Authentication

Users can register with username, email, password, full name, bio, and optional profile picture. After registration, users can log in and access their profile, where they can edit their information. Profile pages show user details, posts, and friendship status.

Friendships

The friendship system allows users to send friend requests, accept or decline requests, and manage their friend list. Friendships have different statuses: Pending, Accepted, and Blocked. The feed only shows posts from accepted friends, ensuring privacy.

Posts and Video Sharing

Users can create posts with a title, video URL, and description. The platform works with Google Drive video links. The feed displays posts from friends in reverse chronological order. Users can delete their own posts.

Feedback

Friends can leave private feedback on each other's audition tapes. Feedback is visible only to the post owner on the My Videos page, creating a private space for constructive criticism. This is a core feature of the platform, supporting the use case of actors helping each other improve their self-tapes.

Messaging

A private messaging system allows users to send direct messages to their friends. The inbox shows conversation threads with unread message counts. Users can view full conversations and reply to messages. Messages are marked as read when viewed.

Admin Dashboard

Administrators have access to a dedicated dashboard with two sections: Contact Messages and User Reports. The Contact Messages section shows messages submitted through the contact form. The Reports section shows user-submitted reports and allows admins to remove users from the platform. When a user is removed, all their associated data is deleted to maintain data integrity.

Contact Form

A contact form allows authenticated users to send messages to administrators. This provides a channel for support requests, feature suggestions, or general feedback. Messages are stored in the database and viewable in the admin dashboard.

User Reporting

Users can report other users for inappropriate behavior. Reports include the reporter, the reported user, a reason, and a timestamp. Admins review these reports and can take action by removing users if necessary.

Code Quality and Best Practices

Coding Style and Conventions

The code follows consistent naming conventions throughout. C# naming conventions are used for classes, methods, and variables. Controllers end with "Controller", models use singular nouns, and views match controller and action names. This consistency makes the codebase easier to read and maintain.

Code Comments

Code comments are included where logic is complex or non-obvious. The database context includes comments explaining the purpose of each DbSet. Controller methods have comments describing their purpose and parameters.

Error Handling

Data validation and server-side length checks prevent common errors such as SQL truncation exceptions. User-friendly messages are displayed using TempData when operations complete or when input is invalid. MaxLength attributes on models and maxlength attributes on form inputs work together to enforce data integrity at both the client and server level.

Database Design

The database schema includes eight tables: Users, Friendships, Posts, Messages, ContactMessages, Reports, Feedbacks, and DismissedPosts. The Users table is central, with most other tables referencing it through foreign key columns such as UserId, SenderId, ReceiverId, and ReporterId.

The Friendships table stores each relationship as a single row with User1Id and User2Id, along with a Status column (Pending, Accepted, or Blocked), a RequestedBy column to track who initiated the request, and a BlockedBy column for block actions. This single-row approach avoids data duplication but requires queries to check both columns when looking up a user's friends.

Messages use SenderId and ReceiverId to link two users, with an IsRead flag for tracking unread messages. The DismissedPosts table tracks which posts a user has swiped past on the feed, so they are not shown again. Feedbacks link a sender to a specific post, allowing friends to leave private comments on each other's audition tapes.

The database setup script in the docs folder creates all tables and populates them with dummy data for demonstration purposes. All column lengths are enforced at both the database level (NVARCHAR limits) and the application level (MaxLength attributes and server-side checks).

Documentation and Setup

Setup Instructions

The project includes a README.md in the root directory and a database setup script in the docs folder. The README covers prerequisites, setup steps, demo accounts, API endpoints, and project structure. The database script creates all tables and populates them with dummy data for demonstration purposes. Together, these provide clear instructions on:

- How to create the database in SQL Server Management Studio
- How to run the seed script to populate the database
- How to configure the connection string in appsettings.json
- How to run the application using Visual Studio or `dotnet run`

Technologies Used

- .NET Core MVC
- Entity Framework Core
- SQL Server
- Bootstrap 5
- JavaScript (with jQuery available as a library dependency)

Dependencies

The project dependencies are listed in the .csproj file and are restored automatically when building the project. No external dependencies beyond the standard .NET packages are required.

Challenges Faced

The biggest challenge was moving from the single-table structures in the sample projects to a database with eight related tables and foreign key relationships. The friendship system in particular was difficult to get right — friendships are bidirectional, meaning either User1 or User2 could be the current user, and the status changes over time from Pending to Accepted or Blocked. Writing the LINQ queries to handle this correctly took a lot of trial and error.

Cascading deletes were another area that caused problems. When an admin removes a user, all of that user's posts, messages, friendships, feedback, reports, and dismissed posts need to be

cleaned up first. Missing even one table results in a foreign key constraint error, so this required careful attention to every relationship in the database.

Combining MVC controllers and API controllers in the same project was also something the sample projects didn't cover, since RazorApp and ProductAPI were separate applications. Getting both to share the same database context and authentication setup required some experimentation.

What Would Be Done Differently

If starting again, I would add proper error pages for common issues like 404 and 500 errors rather than relying on the default developer exception page. I would also add pagination to more areas of the application, such as the message inbox and conversation views, which could become slow with a large number of messages. Separating the API controllers into their own project would also make the architecture cleaner as the application grows.

Conclusion

ActTogether covers the main requirements set out in the brief: Web APIs with CRUD operations, authentication, data validation, MVC and Web API integration, a responsive user interface, and documentation. The development approach followed the patterns established in the sample projects from class, extending them where needed to support the social networking features. There is room for improvement in areas like error handling and pagination, but the application is functional and demonstrates the core concepts covered in the module.